Isolation and Protection in Operating Systems

2. Segmentation and Tasks in x86-64

Prof. Dr. Michael Schöttner
Dr. Fabian Ruhland

WiSe 26/27

x86-64: 64-Bit CPU Architecture

Heinrich Heine
Universität Düsseldorf

- x86-64: Original name of AMD's 64-bit extension to x86
  - Sometimes abbreviated as *x64*
  - Later replaced by *AMD64*
- Intel licensed x86-64 from AMD
- Intel 64 is Intel's own label (for the same architecture)
  - Minimal differences in instruction set compared to AMD64
  - Completely different than the own *IA-64* (Itanium) architechture

x86-64: 64-Bit CPU Architecture

Heinrich Heine
Universität Düsseldorf

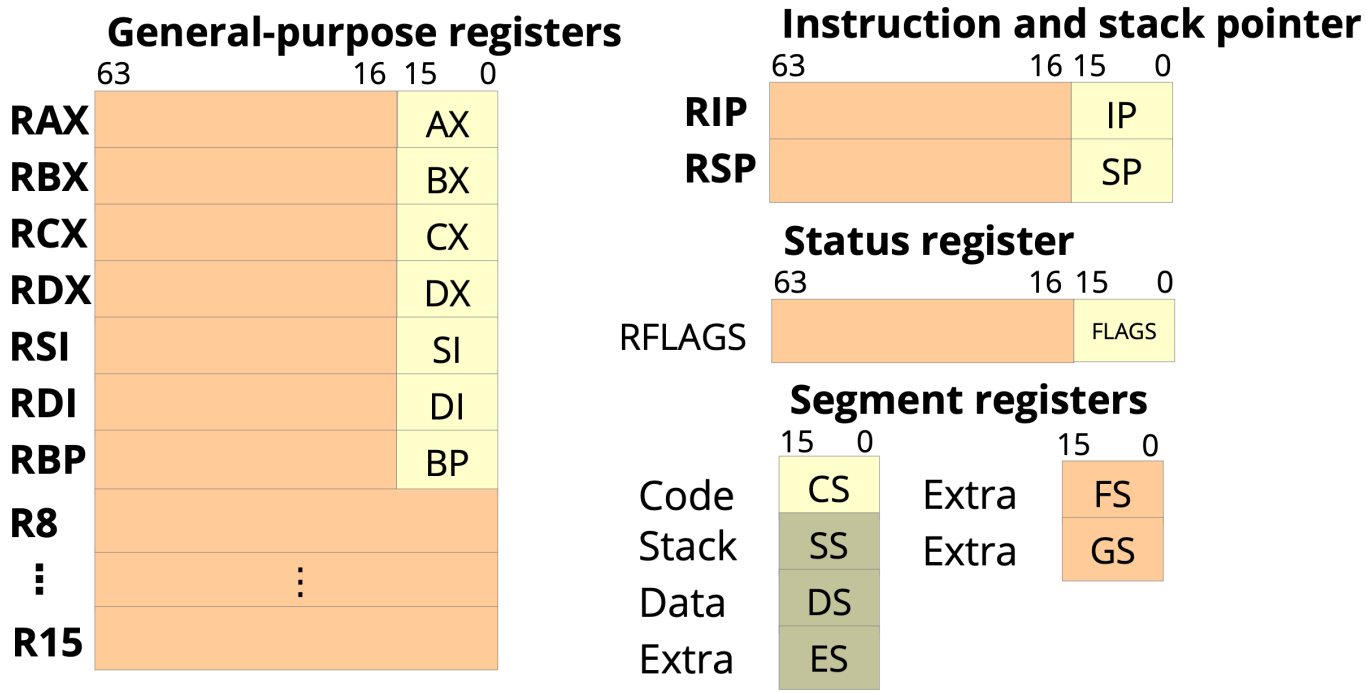
- CPU has 16 general purpose registers (all 64 bit wide)
- Paging must be enabled
- Segmentation strongly restricted

x86-64: CPU modes

Heinrich Heine
Universität Düsseldorf

- Real Mode = 16 bit addressing
- Protected Mode = 32 bit addressing (also called IA-32)
- **Long Mode = 64 Bit addressing (also called IA-32e)**
  - *e* meaning *extension*
  - Used in the Intel manuals
  - To differentiate against IA-64
- (Compatibility Mode)
  - Supporting Protected Mode applications in Long Mode

x86-64: Registers



x86-64: Registers

Heinrich Heine Universität Düsseldorf

Memory-management registers

	15 … 0	63 … 0	31 … 0
TR	TSS sel.	TSS Base Address	TSS Limit
LDTR	LDT sel.	LDT Base Address	LDT Limit
IDTR		IDT Base Address	IDT Limit
GDTR		GDT Base Address	GDT Limit

Details follow ...

Control registers

	63 … 32	31 … 0
CR8		
CR4		
CR3		
CR2		
CR0		

Debug registers

	63 … 32	31 … 0
DR0		
DR1		
DR2		
DR3		
DR6		
DR7		

Model-specific registers (MSRs)

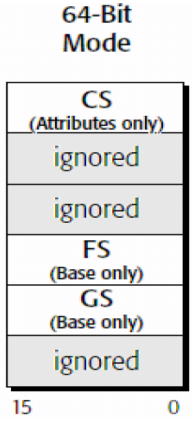
32-Bit Segmentation

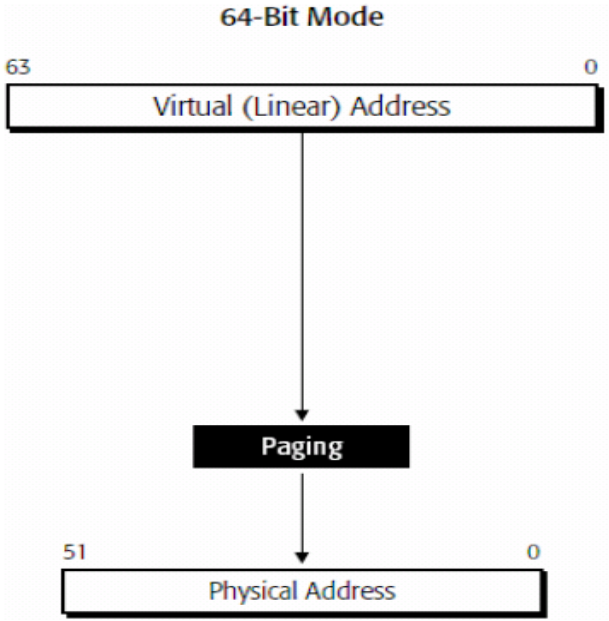
Heinrich Heine
Universität Düsseldorf

- Each segment has a start address and a length
- Segment registers store *segment selectors* = Index into the GDT/LDT
  - LDT was rarely used by operating systems
- A program (in execution) consists of multiple memory segments
  - At least CODE, DATA and STACK
- *Linear address* is segment start address + effective address
  - Effective address = Value in a general purpose register

Long Mode: Segmentation

- 16-bit Selectors
- Start addresses are ignored in almost all segments, except FS and GS
- No limit check; essentially only the ring number in CS remains relevant (= CPL)





Protection

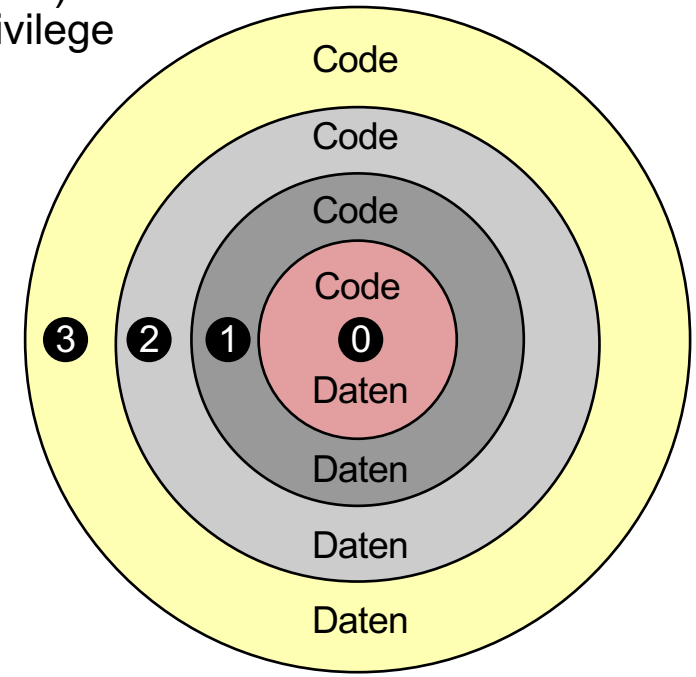
Heinrich Heine
Universität Düsseldorf

- Goals:
  - Protection against unauthorized memory accesses and code execution
  - Protection against kernel crashes or crashes of other processes

- Requirements: Coda and Data...
  - are classified in regard to their trustability
  - and this classification is enforced by the operating system, with the help of the process

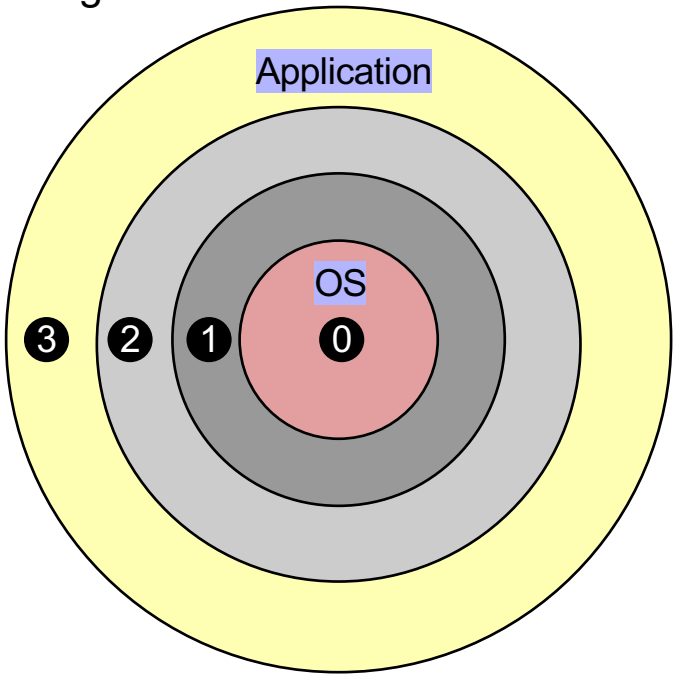
Protection Rings

- Each segment descriptor (Code/Data) has two bits, determining which privilege level the segment is assigned to
  - Up to four privilege levels possible
  - In practice, only two are used



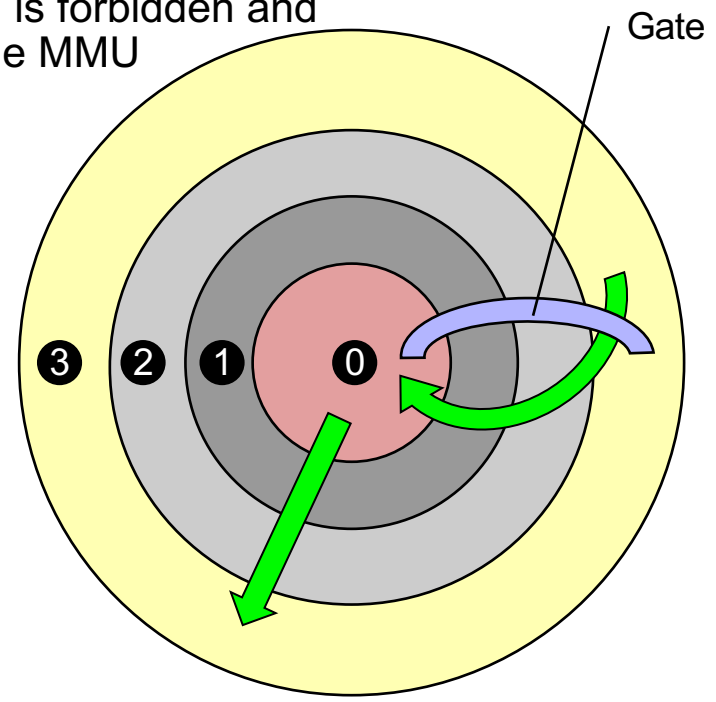
Protection Rings

- Smaller ring numbers grant more privileges
- Ring 3 is meant for applications
  - = User Mode
- Ring 0 is meant for the OS
  - = Kernel/Supervisor Mode



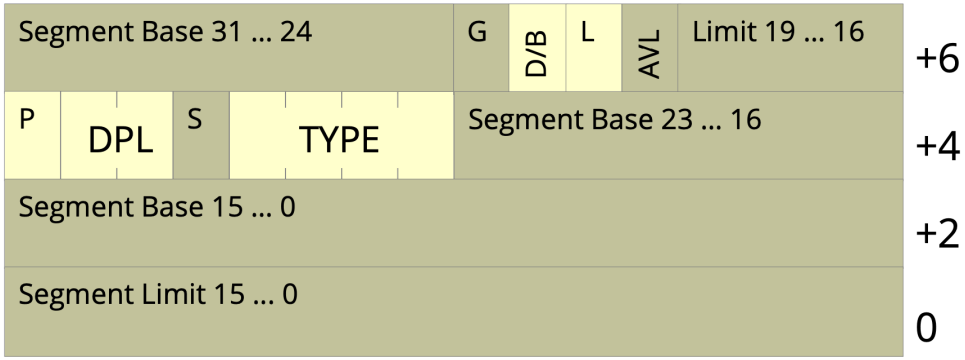
Protection Rings and Gates

- Accessing memory of an inner ring is forbidden and such accesses are prohibited by the MMU
  - General Protection Fault
- Code of an inner ring can only be called via **Gates**
  - Allow direct calls from ring 3 to ring 0
- Memory accesses and calls to outer rings are always allowed



Segment Deskriptors

- Are entries in the Global Descriptor Table (GDT) and allow code protection
  - (IA-32: additional protection via limit (= end of segment))
  - 8 byte per entry (exception: system segments use 16 byte in x86_64)



TYPE
Code/Data
Read/Write
Non-/Conforming

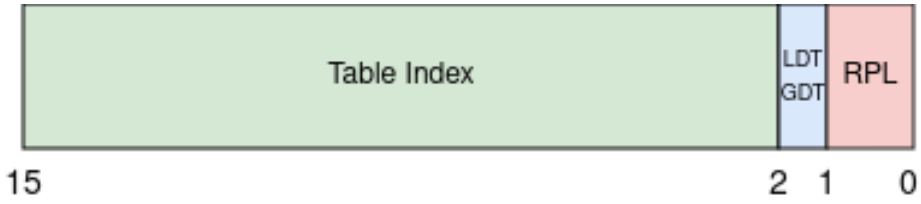
System Segments (Call Gate, Task State Segment)

P	Present Bit
DPL	**Descriptor Privilege Level**
S	System Segment

G	- Granularity
D/B	- 16/32 Bit Seg.
L	- Long Mode aktiv

Segment Selectors

- Stored in 16-bit *Segment Registers*
- Consist of an index into the GDT (13 bit) and the requested privilege level (RPL)



- Privilege Level Abbreviations:
  - Requested Privilege Level (RPL) in a segment register
  - Descriptor Privilege Level (DPL) in the correspoding descriptor
  - Current Privilege Level (CPL) in the current value of the code segment register (bits 0 and 1)
  - IOPL (2 bits) in FLAGS register: Determine the required privilege level to execute I/O instructions
- If RPL > CPL the RPL is used → Privilege level can only be weakened by the RPL

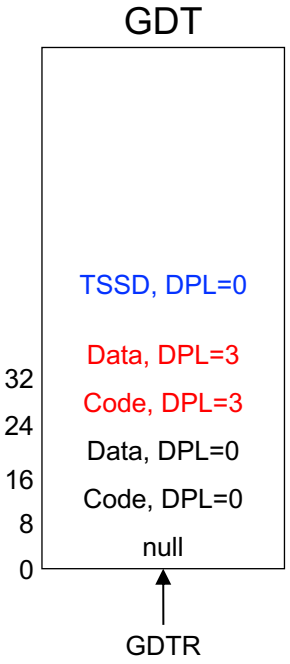
Global Descriptor Table (GDT)

- Only one for the whole OS (independent from the number of cores)
- *Global Descriptor Table Register* (GDTR) points to the GDT

- Typical GDT (also in HeineOS) →
  - DPL = 0: Ring 0 (kernel mode)
  - DPL = 3: Ring 3 (user mode)

- Example: `mov ds, 16` would trigger a *General Protection Fault* if executed in user mode

- TSSD: See next slides



Preview: Protection via Paging

Heinrich Heine
Universität Düsseldorf

- In Long Mode the base address and limit of segments is ignored
  → Segmentation can no longer restrict memory acceses
- Memory protection is done via Paging
  → We can allow/forbid user mode access for each 4 KiB page independently
- Paging allows acces mode restrictions: read only, no execute, etc.
- Paging is discussed in more detail in a later lecture (when we implement it)

Task State Segment (TSS)

Heinrich Heine
Universität Düsseldorf

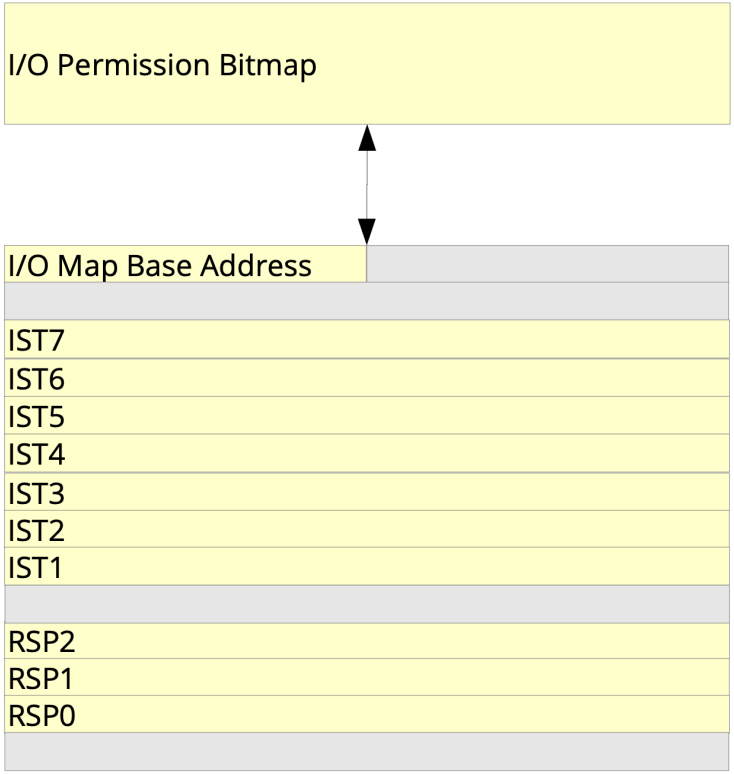
- We need on TSS per CPU core to use kernel/user mode
  - TSS location is described by a TSS descriptor in the GDT
  - The TSS descriptor is referenced by the *Task Register* (TR)

- In 32-Bit Protected Mode the state of a CPU core (all registers, etc.) could be stored inside a TSS (used for hardware supported task switching)
  → Was rarely used and thus abandoned in Long Mode

- In Long Mode the TSS only store Stacks and optionally the I/O Permission Bitmap (see next slide)

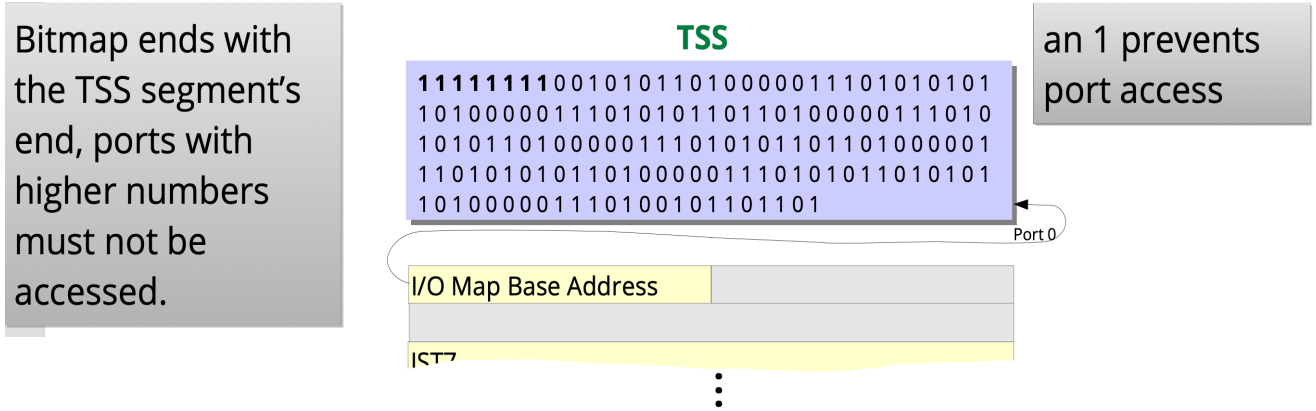
Long Mode: TSS

- IST = Interrupt Stack Table
  - Allows separate stacks for interrupt handlers (optional)
  - IST number can be specified in entries inside the Interrupt-Descriptor Table (IDT)
- RSP0, RSP1, RSP2
  - Stack pointer für rings 0 to 2
  - When the ring changes, the stack is switched and the CPU core finds the right stack pointer in the TSS
  - Most operating systems only use two rings (0 and 3) so that only RSP0 is relevant (applies to HeineOS too)
- RSP3?
  - Missing? → No, it is automatically stored on the stack during a ring change through a call gate or interrupt gate (see later)



Long Mode: I/O-Permission Bitmap

- Access protection for I/O-ports (only little relevance nowadays)
  - I/O privilege level (IOPL) represented by two bits inside the RFLAGS register
  - Port access is granted if CPL ≤ IOPL
  - Otherwise, the I/O permission bitmap is used
    - One bit for each port; if a bit is set, access is denied
- Access protection for memory mapped I/O via page tables



Long Mode: Task Switch

Heinrich Heine
Universität Düsseldorf

- Task switching is done in the kernel, triggered by the timer interrupt

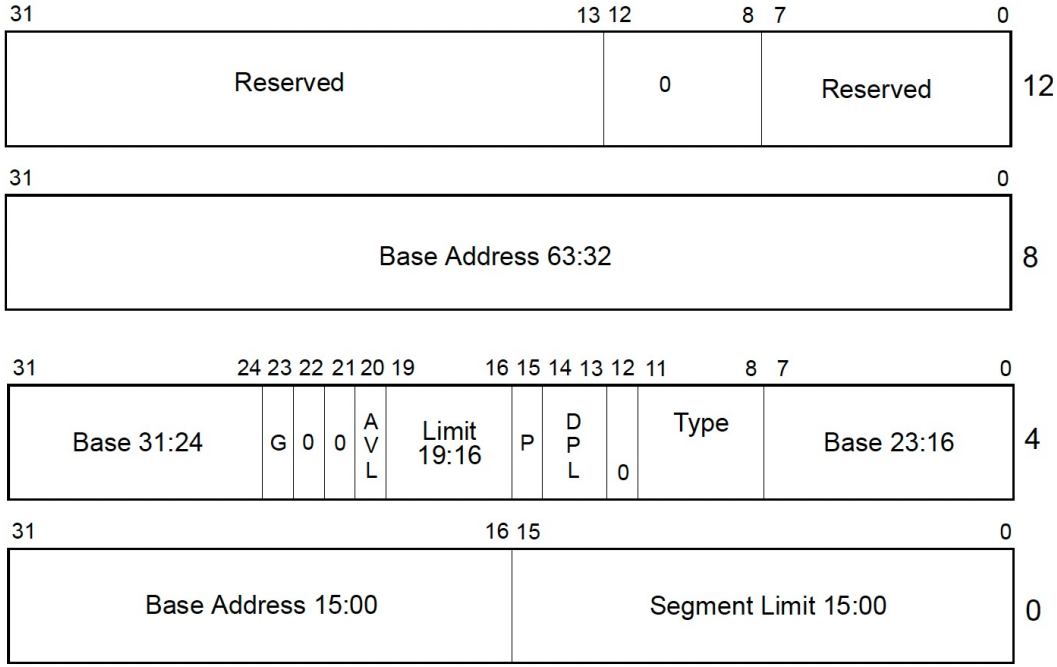
- All registers are stored on the stack of the current thread
  - This is the kernel stack, as the interrupt caused a switch to ring 0
  - Additionally, all values of the TSS are stored on the stack (except RSP0)
    - (In HeineOS, we do not store any TSS values, as we only use RSP0)
  - Finally, RSP0 is saved in the `Thread` structure (residing in the kernel heap)

  - The state of the next thread can be accessed loading the RSP0 from its `Thread` structure

Task State Segment Descriptor (TSSD)

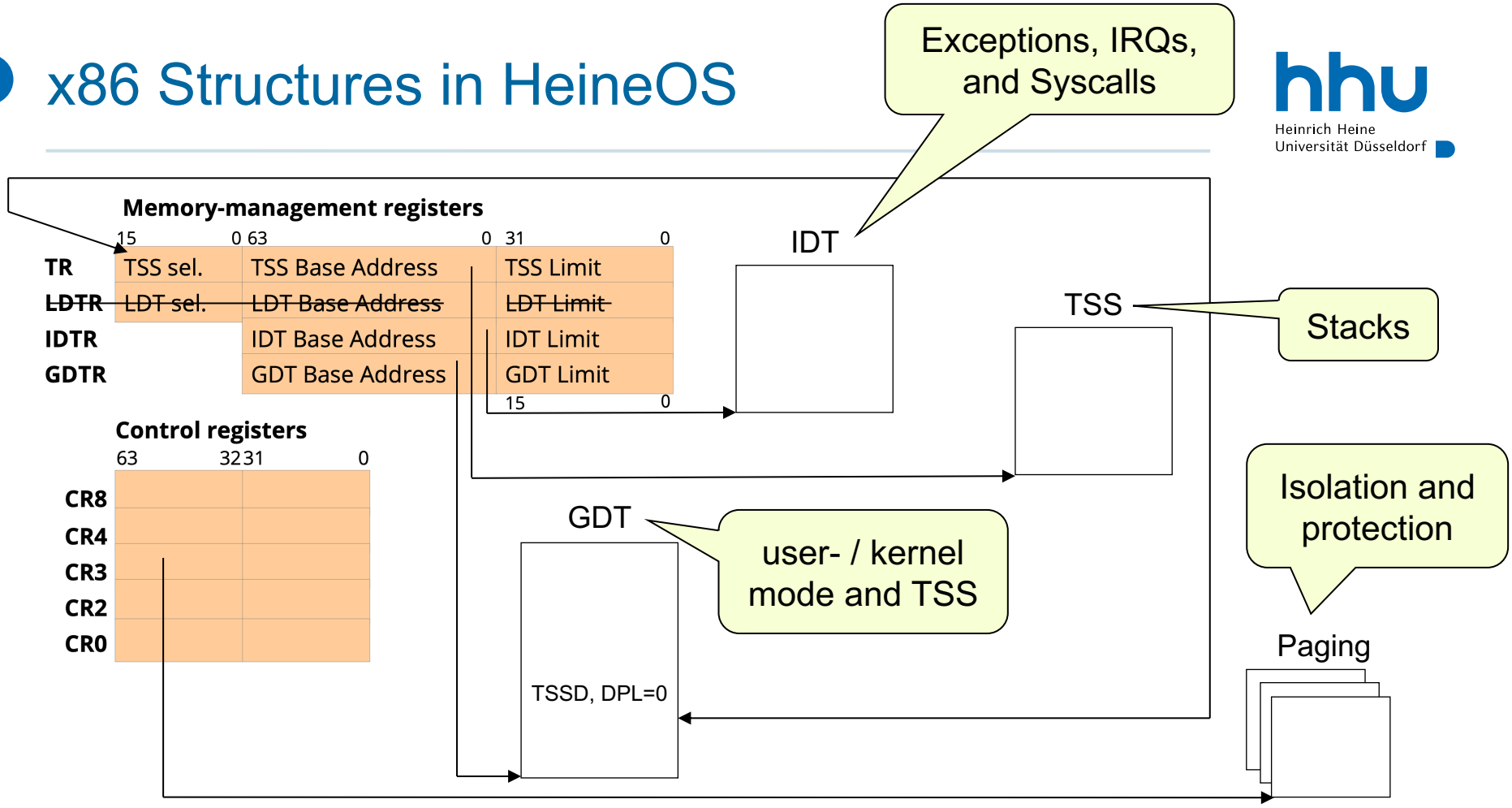
- Stored in GDT
- One entry per core

TSS (or LDT) Descriptor



AVL	Available for use by system software
B	Busy flag
BASE	Segment Base Address
DPL	Descriptor Privilege Level
G	Granularity
LIMIT	Segment Limit
P	Segment Present
TYPE	Segment Type

x86 Structures in HeineOS



Summary

Heinrich Heine
Universität Düsseldorf

- x86-64 architecture is complex
- Rarely used features were removed
  - Segmentation (mostly)
  - Hardware support for task switching
- Still backwards compatible
  - Processor and cores still start up in Real Mode
- Intel *X86s* wanted to simplify the architecture by removing a lot of backward compatibility
  - But, Intel has stopped working ont X86s